

# LC 1203: Sort Items by Group Respecting Dependencies

Google Amazon

## Problem Analysis & Example Setup

| Item | Group | Before |
|------|-------|--------|
| 0    | -1    |        |
| 1    | -1    | 6      |
| 2    | -1    | 5      |
| 3    | 0     | 6      |
| 4    | 0     | 3, 6   |
| 5    | 1     |        |
| 6    | 0     |        |
| 7    | -1    |        |

## Concept & Logical Flow

### Handling Independent Items:

Group -1 means they are individual elements. We can call them group 2, 3, 4 etc. each with 1 member.

"Basic idea to ye he ki jis ka koi group nhi wo apna new individual group bana lege"

### Why a single Kahn's Algorithm fails:

Running Kahn on all items won't simply work because Kahn only guarantees dependency order, but we need to make sure that same group elements are together. If we do a Kahn on all items, then simply 0 1 0 1 0 1 1 1 0 -1 -1 could possibly be generated, breaking group cohesion.

### The "Double Kahn" Approach:

Let's say a member of Group 1 depends on a member of Group 0, then Group 0 must occur before Group 1.

*"Phir ek baar kahn groupwise lagega to decide order of groups."*

*"Phir groupwise kahn lagega for internal order."*

[ Group 0 ] → [ Group 1 ]

**First Kahn:** Establishes external dependency between entire groups.

[ Group 0 ] → [ Group 1 ]

Internal Dependency      Internal Dependency

**Double Kahn:** Now run Kahn inside each group.

*"Ab har ek group ke item me kahn lagana hoga to groupwise pehle items distribute kreng"*

## C++ Implementation

```
class Solution {
public:
    vector<int> kahn(vector<vector<int>>& adj, vector<int>& indeg, vector<int>& nodes){
        queue<int> q;
        for(int n:nodes){
            if(indeg[n]==0){
                q.push(n);
            }
        }
    }
};
```

```

    }
}
vector<int> ans;
while(!q.empty()){
    int curr=q.front();
    q.pop();
    ans.push_back(curr);

    for(auto &it:adj[curr]){
        indeg[it]--;
        if(indeg[it]==0){
            q.push(it);
        }
    }
}

return ans;
}

vector<int> sortItems(int n, int m, vector<int>& group, vector<vector<int>>&
beforeItems) {
    //basic idea to ye he ki
    //jis ka koi group nhi wo apna new individual group bana lege
    //phir ek baar kahn groupwise lagega to decide order of groups
    //phir groupwise kahn lagega for internal order

    for(int i=0;i<n;i++){

        if(group[i]==-1){
            group[i]=m;
            m++;
        }
    }

    vector<vector<int>> itemadj(n);
    vector<int> itemindeg(n,0);

    vector<vector<int>> grpadj(m);
    vector<int> grpinddeg(m,0);

    for(int i=0;i<n;i++){
        for(int x:beforeItems[i]){

            if(group[x]==group[i]){
                itemadj[x].push_back(i);
                itemindeg[i]++;
            }

            else if(group[x]!=group[i]){
                grpadj[group[x]].push_back(group[i]);
            }
        }
    }
}

```

```

        grpinddeg[group[i]]++;
    }
}

}

vector<int> grpnodes;
for(int i=0;i<m;i++){
    grpnodes.push_back(i);
}

vector<int> grouporder=kahn(grpadj,grpinddeg,grpnodes);

if(grouporder.size()!=grpnodes.size()) return {};

//ab har ek group ke item me kahn lagana hoga
// to groupwise pehle items distribute kreng

vector<int> ans;

vector<vector<int>> grphasitems(m);

for(int i=0;i<n;i++){
    grphasitems[group[i]].push_back(i);
}

for(int grp:grouporder){
    vector<int> temp=grphasitems[grp];
    vector<int> grpitemorder=kahn(itemadj,iteminddeg,temp);
    if(grpitemorder.size()!=temp.size()) return {};

    for(int num:grpitemorder){
        ans.push_back(num);
    }
}

return ans;
}
};

```